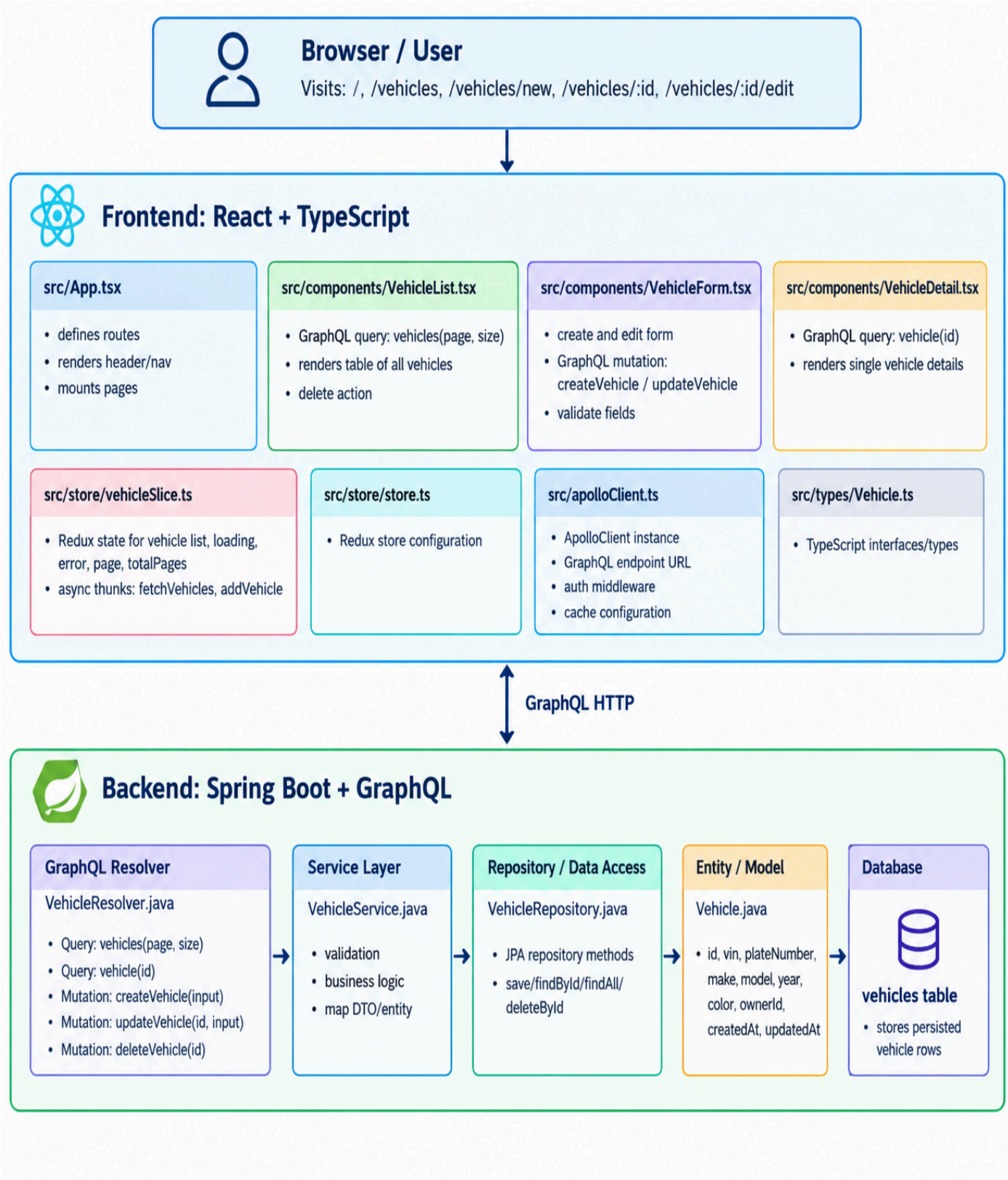
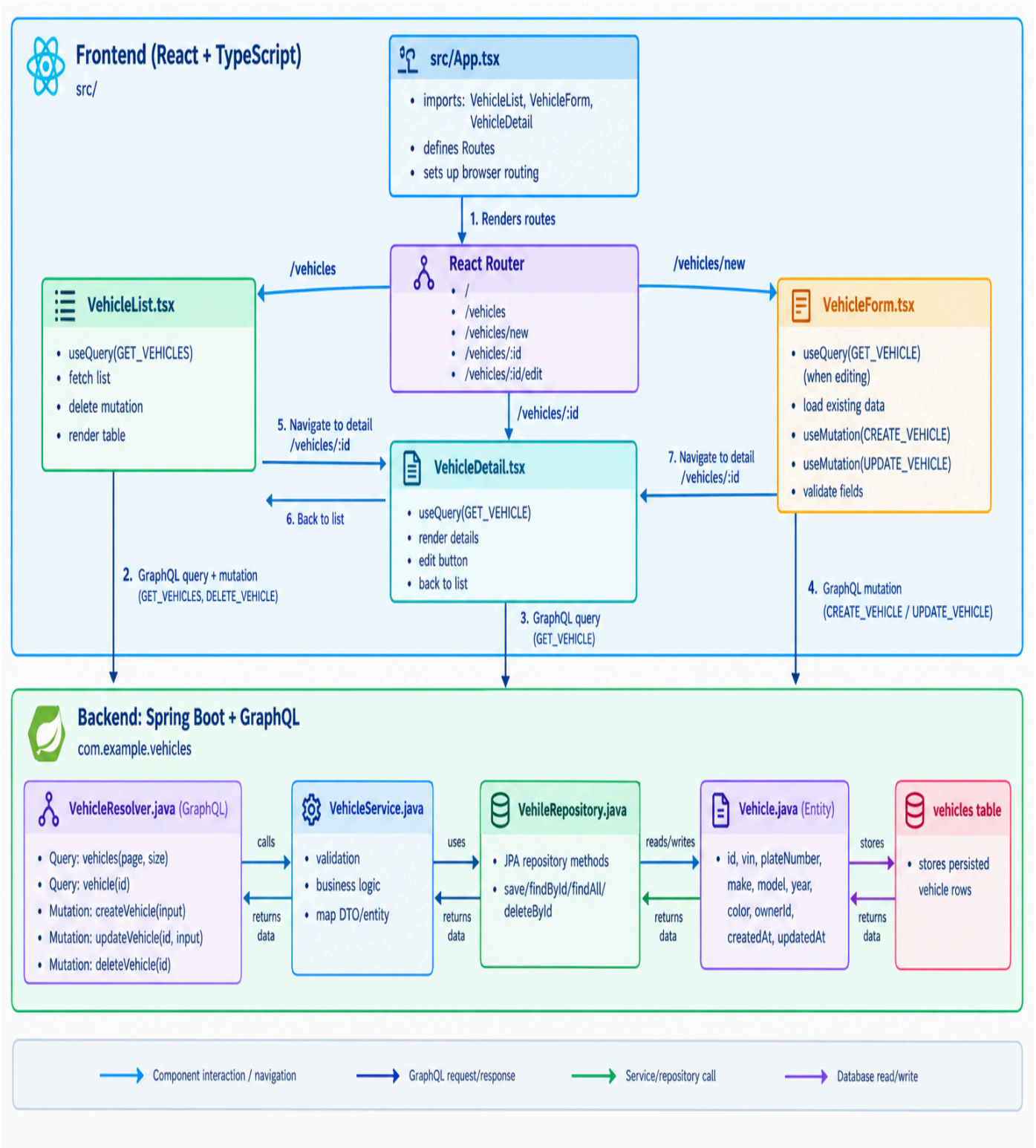
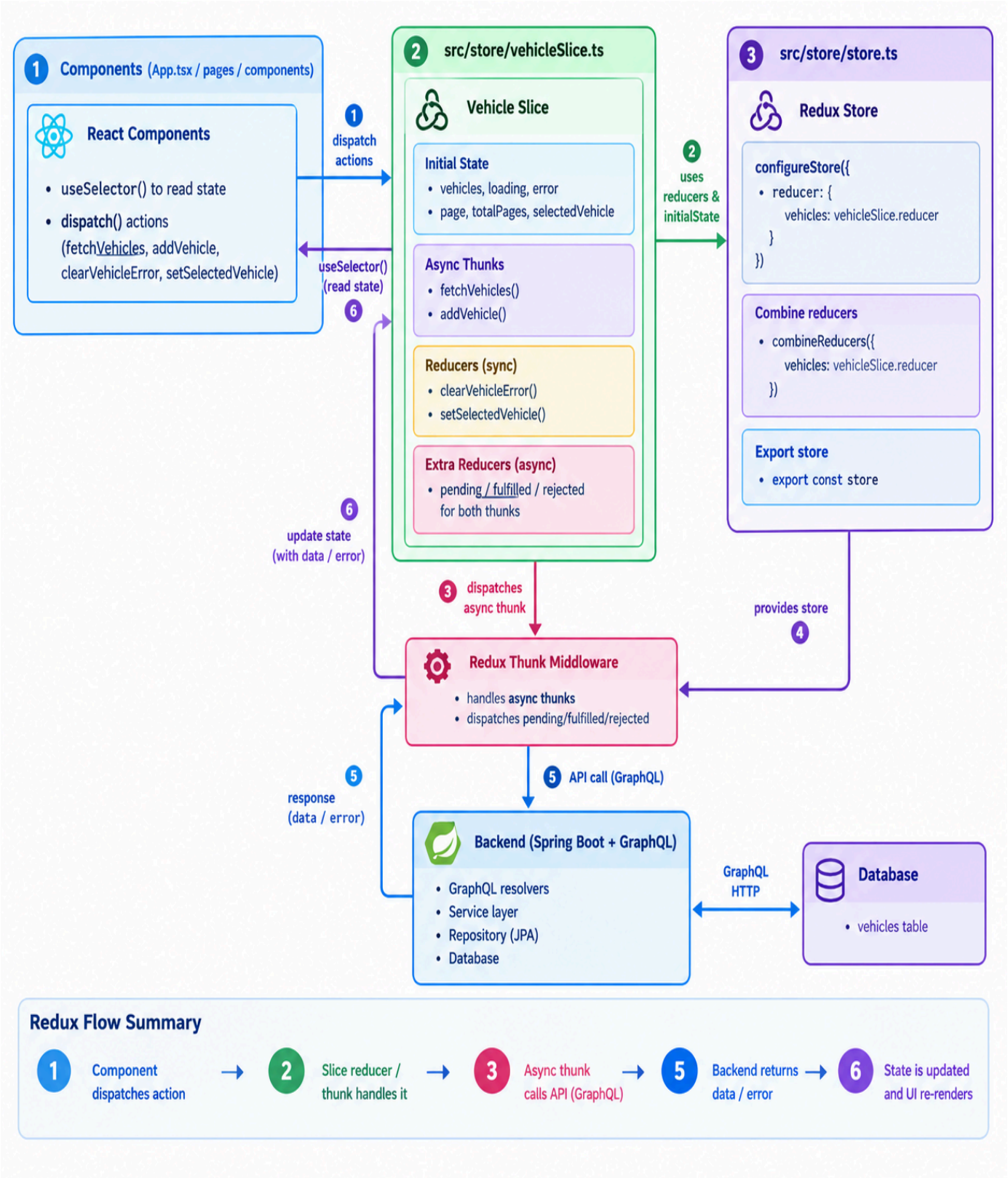

Browser / User
Visits: /, /vehicles, /vehicles/new, /vehicles/:id, /vehicles/:id/edit
Frontend: React + TypeScript
src/App.tsx
• defines routes
• renders header/nav
• mounts pages
src/components/VehicleList.tsx
• GraphQL query: vehicles(page, size)
• renders table of all vehicles
• delete action
src/components/VehicleForm.tsx
• create and edit form
• GraphQL mutation: createVehicle / updateVehicle
• validate fields
src/components/VehicleDetail.tsx
• GraphQL query: vehicle(id)
• renders single vehicle details
src/store/vehicleSlice.ts
• Redux state for vehicle list, loading, error, page, totalPages
• async thunks: fetchVehicles, addVehicle
src/store/store.ts
• Redux store configuration
src/apolloClient.ts
• ApolloClient instance
• GraphQL endpoint URL
• auth middleware
• cache configuration
src/types/Vehicle.ts
• TypeScript interfaces/types
GraphQL HTTP
Backend: Spring Boot + GraphQL
GraphQL Resolver
VehicleResolver.java
• Query: vehicles(page, size)
• Query: vehicle(id)
• Mutation: createVehicle(input)
• Mutation: updateVehicle(id, input)
• Mutation: deleteVehicle(id)
Service Layer
VehicleService.java
• validation
• business logic
• map DTO/entity
Repository / Data Access
VehicleRepository.java
• JPA repository methods
• save/findById/findAll/deleteById
Entity / Model
Vehicle.java
• id, vin, plateNumber, make, model, year, color, ownerId, createdAt, updatedAt
Database
vehicles table
• stores persisted vehicle rows

2. Here is the detailed component interaction diagram for vehicle management application, showing React routing, component interactions, GraphQL queries and mutations, and the Spring Boot backend flow.



Here is the Redux Flow Diagram for the vehicle management application, showing how React components dispatch actions, the Redux slice handles state and async thunks, the store provides state, and GraphQL API responses update the UI.




In Redux, `useSelector()` reads data from the Redux store, while `dispatch()` sends actions to the store.

### 1. `useSelector()`

Read the current state from Redux.

### 2. React renders the UI

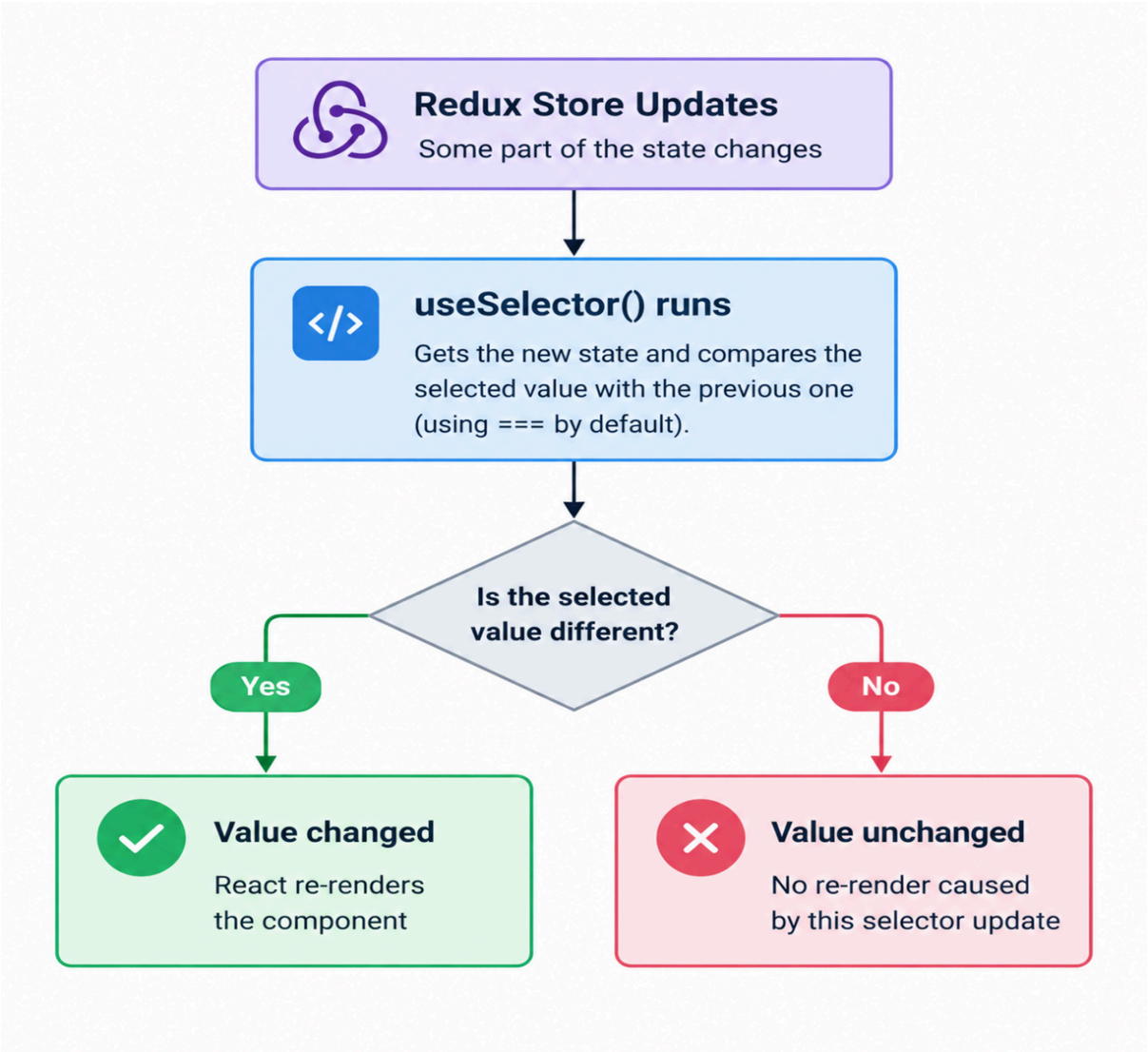
Displays vehicles, loading indicators, or errors.

### 3. `dispatch()` when needed

For example, when the user clicks Refresh or Add Vehicle.

### 4. Redux updates the state

When the state changes, `useSelector()` receives the updated value and React re-renders the component if the selected value changed.



```jsx
function VehicleList() {
    const dispatch = useDispatch();

    // Read state — does not dispatch anything
    const { vehicles, loading, error } = useSelector(
        (state) => state.vehicles
    );

    // Fetch vehicles when the component mounts
    useEffect(() => {
        dispatch(fetchVehicles({ page: 0, size: 10 }));
    }, [dispatch]);

    if (loading) return <p>Loading...</p>;
    if (error) return <p>{error}</p>;

    return (
        <ul>
            {vehicles.map((vehicle) => (
                <li key={vehicle.id}>
                    {vehicle.make} {vehicle.model}
                </li>
            ))}
        </ul>
    );
}
```

`useSelector()` reads the current vehicle list from Redux.

`dispatch(fetchVehicles())` starts fetching vehicles when needed.

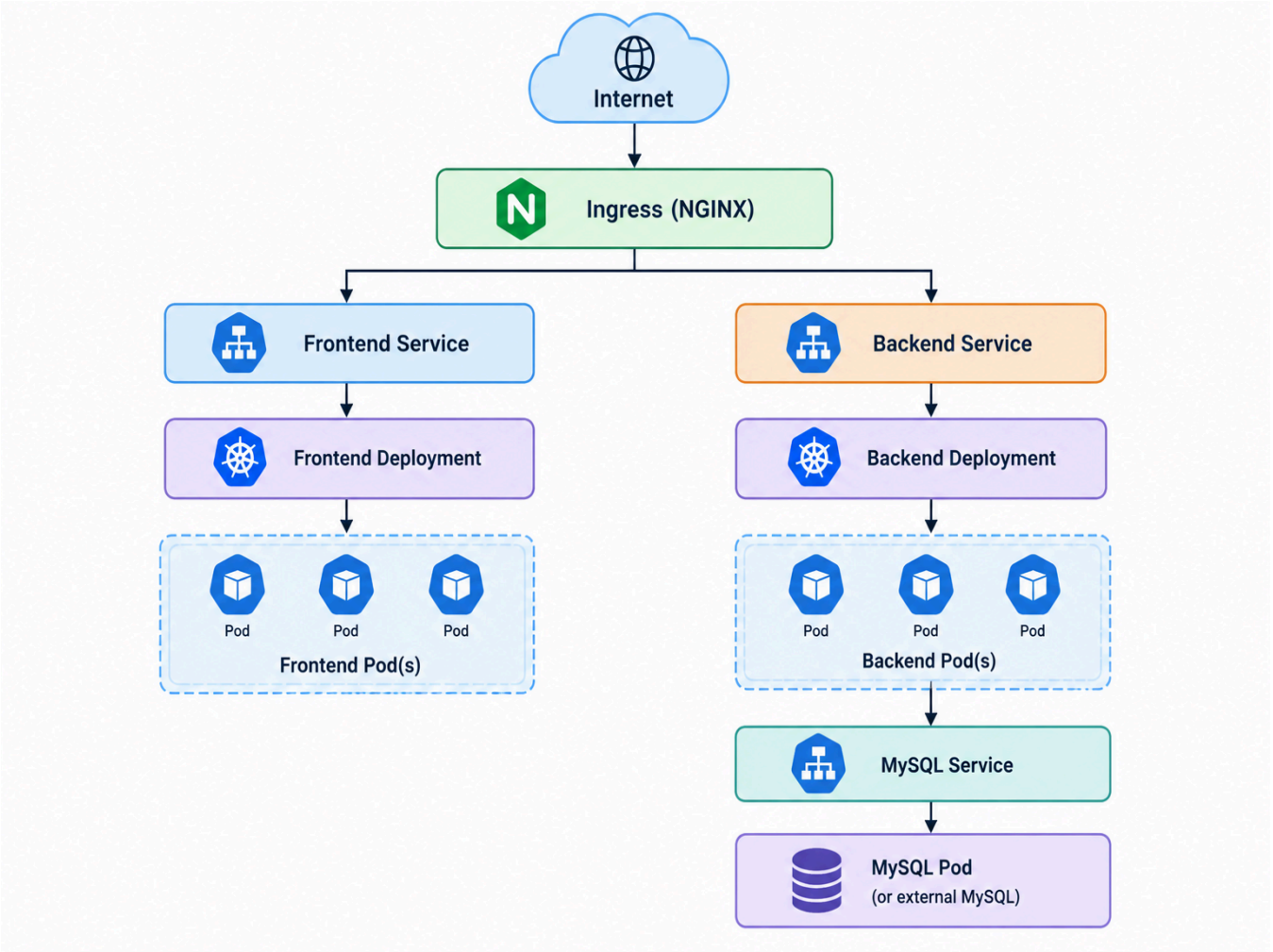
The async thunk calls the API and dispatches `pending`, then `fulfilled` or `rejected`.

The Redux reducer updates the store with the result.

`useSelector()` receives the changed state, and React re-renders the component if the selected value changed.

---

Kubernetes architecture diagram showing how Internet traffic reaches the frontend and backend through NGINX Ingress, and how the backend connects to MySQL.




Dockerfile for backend (Spring Boot Java): **Dockerfile**

Dockerfile for frontend (React + TypeScript + Vite): **Dockerfile**

Add NGINX config for SPA routing : **nginx.conf**

- In Kubernetes, the browser calls frontend service
- Frontend should talk to backend through the backend service name, not localhost
- In a production app, the frontend usually uses a backend URL such as:
  - http://backend.default.svc.cluster.local:8080/graphql
- Or you can make the frontend use `/graphql` via NGINX proxy if you configure it

Docker Hub push steps:

Build backend image:
**cd backend**
**docker build -t your-dockerhub-user/vehicle-backend:1.0.0 .**

**docker push your-dockerhub-user/vehicle-backend:1.0.0**

Build frontend image:
**cd frontend**
**docker build -t your-dockerhub-user/vehicle-frontend:1.0.0 .**
**docker push your-dockerhub-user/vehicle-frontend:1.0.0**

Kubernetes namespace
**k8s/namespace.yaml**

apiVersion: v1
**kind: Namespace**
metadata:
  name: **vehicle-app**

**kubectl apply -f k8s/namespace.yaml**

### Backend Kubernetes deployment

This deployment assumes your Spring Boot app reads env variables like:

- SPRING_DATASOURCE_URL
- SPRING_DATASOURCE_USERNAME
- SPRING_DATASOURCE_PASSWORD

k8s/backend-deployment.yaml

### Frontend Kubernetes deployment

k8s/frontend-deployment.yaml

Ingress for frontend + backend
This is the simplest ingress for exposing both apps.

**k8s/ingress.yaml**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: vehicle-ingress
  namespace: vehicle-app
  annotations:
    kubernetes.io/ingress.class: "nginx"
spec:
  rules:
    - host: app.local
      http:
        paths:
```

```
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend
            port:
              number: 80
  - host: api.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: backend
            port:
              number: 8080
```

Then add host entries to /etc/hosts on your machine (for local cluster testing):
sudo nano /etc/hosts

127.0.0.1 app.local
127.0.0.1 api.local

9. Horizontal Pod Autoscaler

This gives auto-scaling based on CPU.

**k8s/backend-hpa.yaml**

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: vehicle-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2
  maxReplicas: 6
```

```
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Frontend HPA:

k8s/frontend-hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: frontend-hpa
  namespace: vehicle-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: frontend
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Ingress controller for Kubernetes
Install NGINX ingress controller:

**kubectl apply -f**
**https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.11.2/deploy/static/provider/cloud/deploy.yaml**

MySQL in Kubernetes

If the database should also run inside the cluster for a simple dev environment, use a MySQL StatefulSet/Service.
Example MySQL Service:

**k8s/mysql-service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: vehicle-app
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
  clusterIP: None
```

Then **k8s/mysql-deployment.yaml**

**Apply everything:**
**kubectl apply -f k8s/namespace.yaml**
**kubectl apply -f k8s/mysql-service.yaml**
**kubectl apply -f k8s/mysql-deployment.yaml**
**kubectl apply -f k8s/backend-deployment.yaml**
**kubectl apply -f k8s/frontend-deployment.yaml**
**kubectl apply -f k8s/backend-hpa.yaml**
**kubectl apply -f k8s/frontend-hpa.yaml**
**kubectl apply -f k8s/ingress.yaml**

**Check status:**
**kubectl get pods -n vehicle-app**
**kubectl get svc -n vehicle-app**
**kubectl get ingress -n vehicle-app**
**kubectl get hpa -n vehicle-app**

**14. Accessing the app**

**If ingress is working:**

- **Frontend: http://app.local**
- **Backend GraphQL: http://api.local/graphql**

**If the cluster is remote or cloud-based, replace localhost with your ingress host or external DNS.**

**15. Important production notes**

- **Use Secrets instead of plain text env values**
- **Use PersistentVolumeClaim for MySQL**
- **Add Liveness and Readiness probes**
- **Use resource requests/limits**
- **Use a proper TLS certificate for ingress**
- **Add cluster autoscaler if you want node auto-provisioning**
- **Externalize database for production**

build-and-deploy.sh

```
#!/bin/bash

set -e

DOCKER_USER="your-dockerhub-user"

echo "Building backend..."
cd backend
docker build -t ${DOCKER_USER}/vehicle-backend:1.0.0 .
docker push ${DOCKER_USER}/vehicle-backend:1.0.0

echo "Building frontend..."
cd ../frontend
docker build -t ${DOCKER_USER}/vehicle-frontend:1.0.0 .
docker push ${DOCKER_USER}/vehicle-frontend:1.0.0

echo "Applying Kubernetes manifests..."
cd ../k8s
kubectl apply -f namespace.yaml
kubectl apply -f secret.yaml
kubectl apply -f mysql-pvc.yaml
kubectl apply -f mysql-deployment.yaml
kubectl apply -f mysql-service.yaml
kubectl apply -f backend-deployment.yaml
kubectl apply -f backend-service.yaml
kubectl apply -f backend-hpa.yaml
kubectl apply -f frontend-deployment.yaml
kubectl apply -f frontend-service.yaml
kubectl apply -f frontend-hpa.yaml
kubectl apply -f ingress.yaml

echo "Deployment complete."
```

**Run Commands:**

```
chmod +x build-and-deploy.sh
./build-and-deploy.sh
```

The ingress acts as your gateway (a reverse proxy and load balancer). Here is the path and how to see which pod handled a request.

**Browser → ingress-nginx (localhost:80)**
**├─ Host app.local → Service "frontend" → frontend pods (2)**
**└─ Host api.local → Service "backend" → backend pods (2) → Service "mysql" → mysql pod (1)**

- The ingress picks the host by name, then sends each request to one of the pods behind that Service, round-robin by default. ingress-nginx sends traffic directly to pod IPs (from the Service's endpoints), not through the Service's virtual IP.
- Consecutive requests can land on different backend pods. Since you use stateless JWT, that works fine.
- MySQL is not exposed through the ingress. Only the backend pods talk to it, through the `mysql` Service.

### 1. See all pods and their IPs:

```
Shell
kubectl get pods -n vehicle-app -o wide
kubectl get endpoints backend frontend mysql -n vehicle-app
```

The endpoints list the pod IPs that receive traffic.

## 2. Logs of every backend pod

Each pod keeps its own log. Follow them all at once, with the pod name as a prefix:

```
Shell
kubectl logs -l app=backend -n vehicle-app -f --prefix --tail=20
--max-log-requests=5
```

**kubectl logs -f <backend-pod-name> -n vehicle-app**

By default Spring doesn't log each request, so you may see nothing for a normal login. Make it visible by adding an env var to the backend deployment:

**- name: LOGGING_LEVEL_ORG_SPRINGFRAMEWORK_WEB**

**value: DEBUG**

or `LOGGING_LEVEL_ORG_SPRINGFRAMEWORK_SECURITY: DEBUG` to see the auth flow. Use these only while testing, since they are noisy.

## 3. Prove load balancing

Return the pod name in a response or hit an endpoint repeatedly while tailing the logs:

**for i in {1..10}; do curl -s -o /dev/null -w "%{http_code}\n" http://api.local/actuator/health; done**

(`/actuator/health` must be allowed in your security config, or you'll see 403, which still gets logged by the ingress.)

## 4. The ingress controller's own access log

This shows every request and which backend pod IP it was sent to (the `[vehicle-app-backend-8080]` and the upstream address):

```
Shell
kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller
-f --tail=20
```

Match the upstream IP to a pod in `kubectl get pods -o wide`. This is the most direct way to see

Match the upstream IP to a pod in `kubectl get pods -o wide`. This is the most direct way to see which pod got hit.

## 5. MySQL

There is one MySQL pod, so:

```
Shell
kubectl logs deploy/mysql -n vehicle-app -f
```

MySQL doesn't log queries by default. To see what the backend is doing:

```
Shell
# who is connected right now
```

```
kubectl exec -it deploy/mysql -n vehicle-app -- mysql -uroot -p -e "SHOW PROCESSLIST;"



# check the data

kubectl exec -it deploy/mysql -n vehicle-app -- mysql -uappuser -p vehicledb -e "SELECT id, username, role FROM users;"
```

To watch every query (dev only), enable the general log temporarily:

bash

```
Shell
kubectl exec -it deploy/mysql -n vehicle-app -- mysql -uroot -p -e "SET GLOBAL general_log='ON'; SET GLOBAL log_output='TABLE';"

kubectl exec -it deploy/mysql -n vehicle-app -- mysql -uroot -p -e "SELECT event_time, argument FROM mysql.general_log ORDER BY event_time DESC LIMIT 10;"

kubectl exec -it deploy/mysql -n vehicle-app -- mysql -uroot -p -e "SET GLOBAL general_l
```

Alternatively, `spring.jpa.show-sql: true` is already in your yml, so each backend pod logs the SQL it runs.

## 6. Frontend pods

```
Shell
kubectl logs -l app=frontend -n vehicle-app --prefix -f
```

The frontend only serves static files, so it logs one line per file request, not API calls. The API calls go from the browser directly to `api.local` (you can see them in the browser's Network tab).

## Handy extras

```
Shell
kubectl top pods -n vehicle-app                                    # CPU and memory per pod
```

```
kubectl exec -it <backend-pod> -n vehicle-app -- sh      # shell inside a pod

kubectl get events -n vehicle-app --sort-by=.lastTimestamp    # recent cluster events
```

For a nicer multi-pod log view, install `stern` (`brew install stern`) and run `stern backend -n vehicle-app`, which colours each pod's output.

---

1. How to build the updated Java backend Docker image.
2. How to tag the Docker image with version `1.0.8`.
3. How to push the updated image to Docker Hub or my container registry.
4. How to update the Kubernetes backend Deployment YAML with the new image version.
5. How to apply the updated Kubernetes YAML files using `kubectl`.
6. How to safely remove the old backend pods and create new pods using the updated image.
7. How to perform a rolling restart or rollout without causing unnecessary application downtime.
8. How to check whether the new pods are running successfully.
9. How to verify that the updated backend application is working correctly.
10. How to check the deployment status, pod logs, services and endpoints.

## Part 1: Deploy backend 1.0.8 <Any version>

### 1. Build the jar

Shell
```
./mvnw clean package -DskipTests        # or ./gradlew clean build -x test

unzip -l target/*.jar | grep db/migration    # optional: confirm migrations are inside
```

### 2. Build and push the image with the new tag

Shell
```
docker build -t gayanscorpio/vehicle-backend:1.0.8 .

docker push gayanscorpio/vehicle-backend:1.0.8
```

**3. Update the YAML**

In `k8s/backend-deployment.yaml`:

yaml

```
None
image: gayanscorpio/vehicle-backend:1.0.8
```

**4. Apply and watch the rollout**

```
Shell
cd k8s

kubectl apply -f backend-deployment.yaml

kubectl rollout status deploy/backend -n vehicle-app

kubectl get pods -n vehicle-app -l app=backend
```

Kubernetes starts new pods first and removes the old ones once the new ones are ready, so there is no downtime. With the patient probes in place, the rollout can take a few minutes.

**5. Verify**

```
Shell
kubectl get deploy backend -n vehicle-app -o
jsonpath='{.spec.template.spec.containers[0].image}'; echo

kubectl logs -l app=backend -n vehicle-app --prefix --tail=30 |
grep -E "Started|ERROR"
```

**If something goes wrong, roll back:**

```
Shell
kubectl rollout undo deploy/backend -n vehicle-app
```

Then fix the YAML tag to match, or `kubectl apply` will put it back.

**Rules of thumb**

- Always use a new tag. Reusing a tag won't trigger a rollout.

- Keep the YAML and the running version in sync. If you ever use: `kubectl set image`, update the file too.
- If you only changed environment values or secrets (not the image), restart with `kubectl rollout restart deploy/backend -n vehicle-app`.

## Part 2: Remove pods

**Delete the Deployments and Services, keep the database volume**

```
Shell
kubectl delete deploy,svc,ingress,hpa --all -n vehicle-app
```

The PVC and secret remain, so MySQL data survives. Re-create with `kubectl apply -f` on each file.

**Option B: wipe everything, including the database**

```
Shell
kubectl delete namespace vehicle-app

kubectl get ns          # wait until it disappears
```

This deletes the pods, services, ingress, secret and the PVC, so all MySQL data is lost (you would need to register users again). Re-create with the step-by-step order from my earlier reset message: namespace, secret, PVC, MySQL, backend, frontend, ingress.

**Restart pods without deleting anything**

bash

```
Shell
kubectl rollout restart deploy/backend -n vehicle-app

kubectl rollout restart deploy --all -n vehicle-app
```

**Remove only a stuck pod**

bash

```
Shell
kubectl delete pod <pod-name> -n vehicle-app
```

```
kubectl delete pod <pod-name> -n vehicle-app --force
--grace-period=0    # if it hangs in Terminating
```

**Comparing ways to protect secrets in a yaml file.**

Kubernetes Secrets are only base64-encoded, not encrypted, so anyone who can read the YAML or the Secret can decode it (`echo ... | base64 -d`). You have three practical options, from simplest to most robust.

## Option 1: Don't put passwords in YAML at all (simplest)

Create the secret from the command line and keep `secret.yaml` out of Git:

bash

```
Shell
kubectl create secret generic vehicle-secrets -n vehicle-app \
  --from-literal=MYSQL_ROOT_PASSWORD='your-root-pw' \
  --from-literal=MYSQL_DATABASE='vehicledb' \
  --from-literal=MYSQL_USER='appuser' \
  --from-literal=MYSQL_PASSWORD='your-app-pw' \
  --from-literal=SPRING_DATASOURCE_USERNAME='appuser' \
  --from-literal=SPRING_DATASOURCE_PASSWORD='your-app-pw'
```

Use single quotes so special characters like `$` and `!` survive. To avoid the passwords landing in your shell history, read them from a local file that is in `.gitignore`:

```
Shell
kubectl create secret generic vehicle-secrets -n vehicle-app
--from-env-file=.env.k8s
```

Add `secret.yaml`, `mysql-secret.yaml` and `.env.k8s` to `.gitignore`, and keep a `secret.example.yaml` with fake values in the repo.

### Option 2: Encrypt the YAML so it is safe in Git (SOPS + age)

SOPS encrypts only the values and leaves the keys readable, so the file still diffs nicely.

bash

```
Shell
brew install sops age

age-keygen -o ~/.config/sops/age/keys.txt        # prints your
public key: age1...
```

Create `.sops.yaml` in the k8s folder:

yaml

```
None
creation_rules:

  - path_regex: secret\.yaml$

    encrypted_regex: ^(data|stringData)$

    age: age1yourpublickeyhere
```

Encrypt and deploy:

bash

```
Shell
sops -e -i secret.yaml                           # encrypts in
place

sops -d secret.yaml | kubectl apply -f -         # decrypt and
apply in one step

sops secret.yaml                                 # edit it
(decrypts in your editor, re-encrypts on save)
```

Commit the encrypted file. **Back up `keys.txt` and never commit it**, because without it nobody can decrypt the file.

### Option 3: Sealed Secrets (good for a real cluster, GitOps style)

A controller in the cluster holds the private key, and you commit a `SealedSecret` that only that cluster can decrypt.

bash

```
Shell
brew install kubeseal

kubectl apply -f
https://github.com/bitnami-labs/sealed-secrets/releases/latest/do
wnload/controller.yaml



kubectl create secret generic vehicle-secrets -n vehicle-app \

  --from-literal=MYSQL_PASSWORD='your-app-pw' --dry-run=client -o
yaml \

  | kubeseal --format yaml > sealed-secret.yaml



kubectl apply -f sealed-secret.yaml
```

Check the Sealed Secrets release page for the current install URL and version before running it. `sealed-secret.yaml` is safe to commit. The catch is that if you delete the cluster (or the controller's key), the sealed file can't be decrypted, so back up the controller key.

### Other layers worth knowing

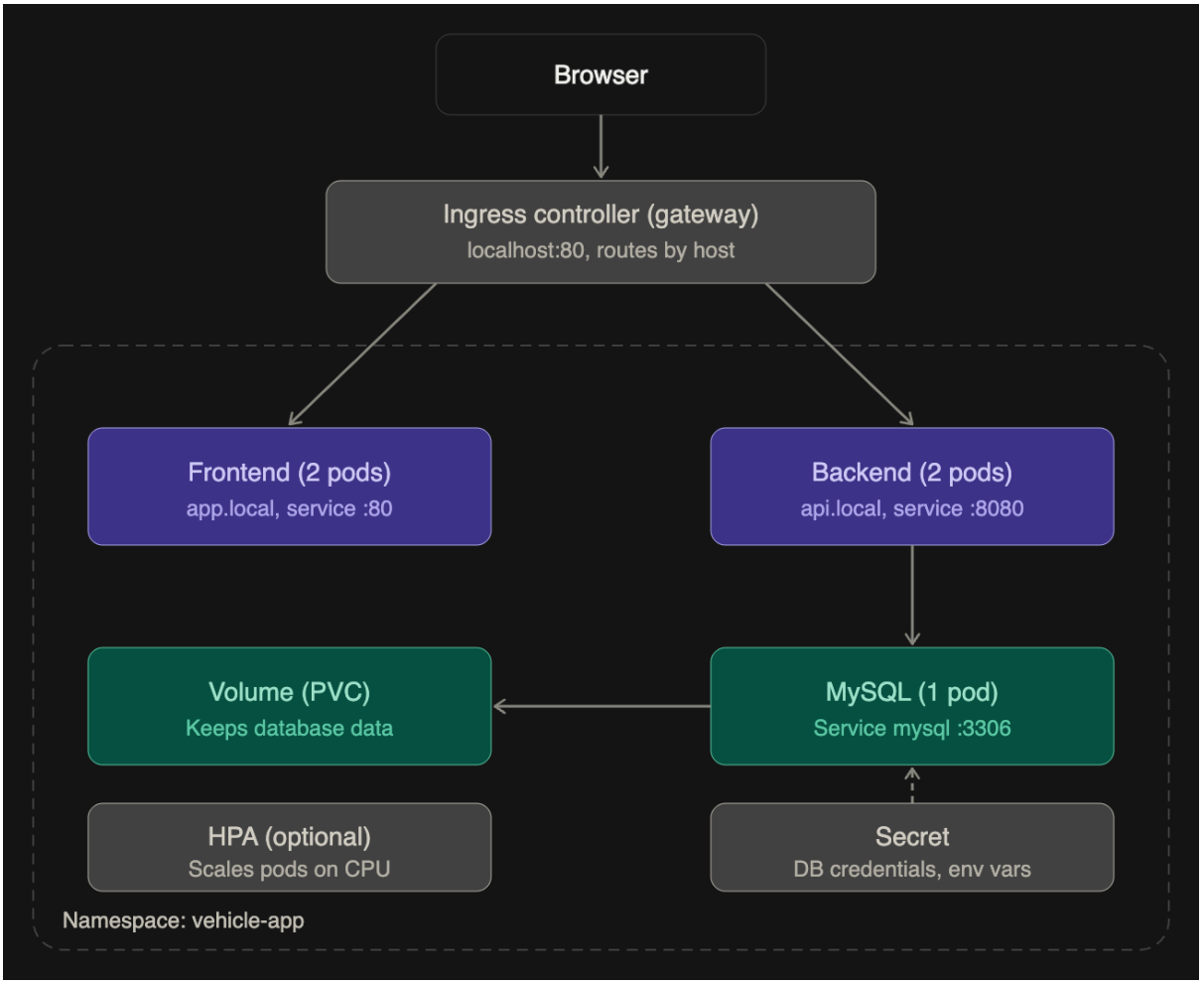
- **Encryption at rest for etcd.** Option 1 to 3 protect the file, but the Secret in the cluster is still stored unencrypted in etcd by default. On managed clusters (EKS, GKE, AKS) enable this in the provider settings. It isn't needed on Docker Desktop.
- **External secret stores.** For production, use AWS Secrets Manager, Azure Key Vault, GCP Secret Manager or HashiCorp Vault with the External Secrets Operator, so no secret lives in Git at all.
- **RBAC.** Limit who can run `kubectl get secret` in the namespace, since anyone who can read a Secret can decode it.

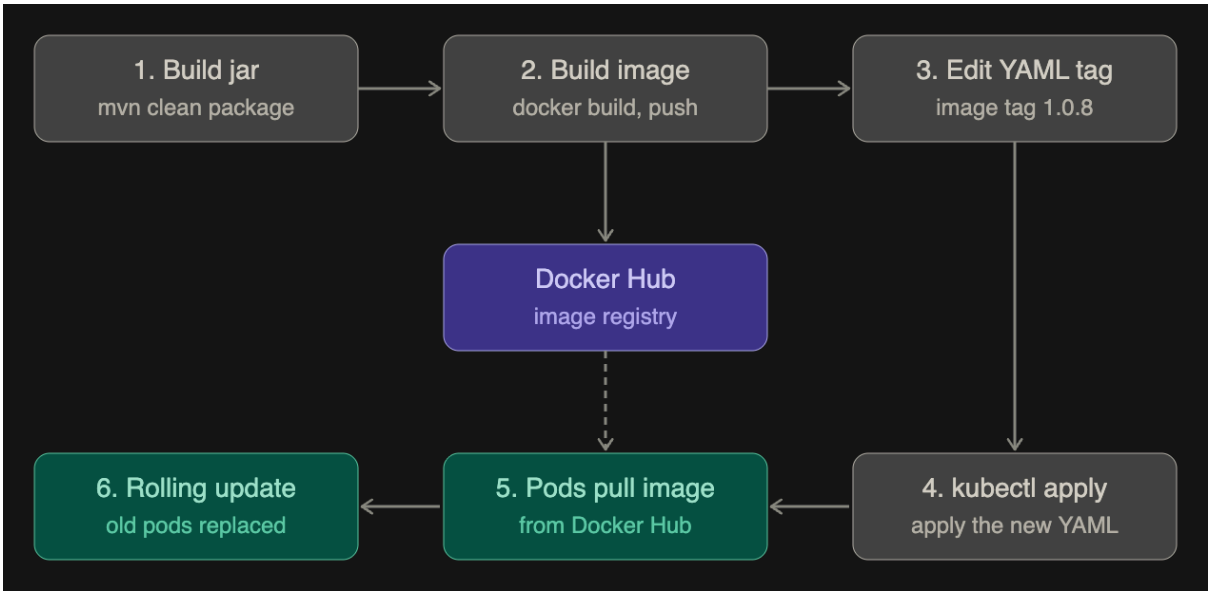
### Fix the app side too

- Your `application.yml` had a real-looking password (`password: xxxxxxx`). Replace it with the placeholders I showed earlier (`${SPRING_DATASOURCE_PASSWORD:}`) so no password is in the jar or Git.
- If any password was ever committed, change it, since it stays in Git history.
- Use strong, unique values, not `apppass123`.
- Your JWT signing secret (used by `JwtUtil`) belongs in the Secret too, passed as an env var, not hardcoded.

**My recommendation:** use Option 1 now (quick, nothing sensitive in Git). Switch to SOPS (Option 2) if you want the encrypted file in the repo, or Sealed Secrets if you later run a real cluster with GitOps.

## Containerised Deployment of the Vehicle Management System on Kubernetes






The dashed arrow in the second diagram is the image pull. Docker Hub is only read when the new pods start. Kubernetes does not watch it, which is why step 3 (a new tag in the YAML) is what triggers the rollout.

What each layer does at runtime

- Ingress controller: the single entry point. It matches the `Host` header (`app.local` or `api.local`) and forwards to the right Service.
- Services: stable names and load balancing. The `backend` and `frontend` Services spread requests across their pods, and `mysql` is the name your JDBC URL resolves.
- Pods: frontend and backend are stateless, so they can scale to 2 or more. MySQL is stateful, so it stays at 1 pod with the `Recreate` strategy.
- Probes (not drawn): the backend's startup, liveness and readiness probes decide when a pod gets traffic. A pod only receives requests once it passes readiness.
- PVC: the database files live outside the pod, so restarting MySQL keeps the data. Deleting the PVC wipes it.
- Secret and HPA: the Secret is read when each pod starts, so after changing it you must restart the pods that use it. The HPA only changes the number of pods.

Login request, end to end

1. The browser sends `POST api.local/api/auth/login`.
2. The ingress picks a backend pod through the `backend` Service.
3. The backend checks the user in MySQL through the `mysql` Service, then returns a JWT.
4. Later requests carry the JWT, and any backend pod can validate it, since no session is stored in the pod.